

FOCUS BEAR

AWS Lambda MCP Architecture Proposal

A secure, scalable way to give AI agents scoped access to Focus Bear tasks.

Client presentation | 11 Sep 2026

Draft for discussion

Recommended approach

Start with a thin, stateless MCP adapter; keep Focus Bear's API and database as the source of truth.

Public edge

API Gateway REST API

- HTTPS endpoint
- rate limits and optional WAF
- response streaming when needed

MCP adapter

AWS Lambda

- validates MCP protocol
- maps tools to API requests
- does not own business data

Focus Bear authority

Existing NestJS API + PostgreSQL

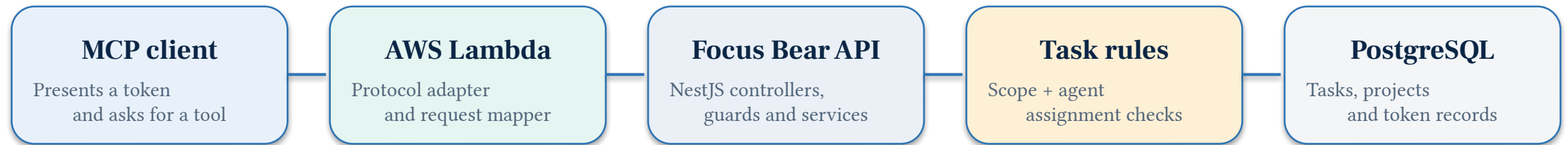
- validates user token and scopes
- preserves agent/task assignment
- applies domain rules

Why this is the right v1

- Fast to ship: reuses the existing external-MCP endpoints and permission checks.
- Low operational overhead: no new database, queue, or always-on service.
- Safe evolution path: add OAuth, more tools, or container hosting only when usage requires it.

It builds on the architecture already in Focus Bear

The repository already has an external MCP API boundary and a Lambda-oriented MCP server prototype.



Existing building blocks

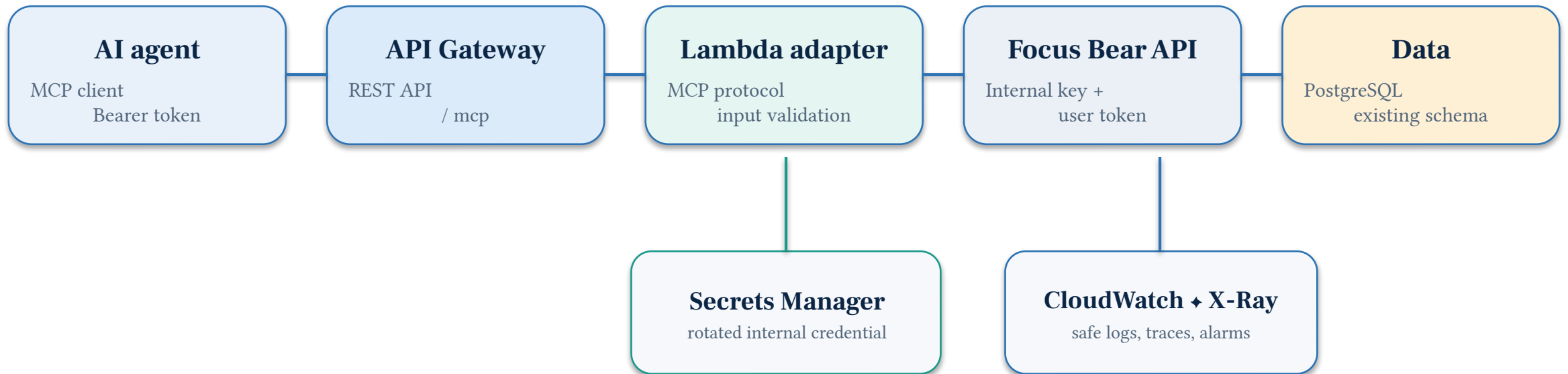
- External MCP task endpoints
- Bearer-token validation and scopes
- Token-to-agent assignment on tasks
- Per-token throttling
- A Lambda streaming prototype

Important v1 cleanup

- Remove development fallback credentials
- Restore real authentication on token issuance
- Make the internal service key a required secret
- Agree an explicit public endpoint / network path

One clear request path, with two trust boundaries

The Lambda layer is deliberately thin; the Focus Bear API remains the policy enforcement point.



Trust boundary 1: public MCP client → AWS

Trust boundary 2: AWS adapter → Focus Bear API

Hosting choices

Recommendation is driven by MCP transport compatibility and low operational burden, not technology novelty.

Recommended

API Gateway REST API +
Lambda

Pros: supports Lambda response streaming; managed edge controls; serverless operations.

Trade-off: slightly more configuration than HTTP API.

Conditional fallback

API Gateway HTTP API +
Lambda

Pros: simpler and often lower cost.

Trade-off: use only if all target MCP clients work with buffered HTTP responses; not the streaming default.

Not for v1

Lambda Function URL

Pros: least infrastructure.

Trade-off: fewer gateway controls and less deliberate public edge; not preferred for a client-facing service.

Later, if needed

ECS / Fargate

Pros: best for persistent connections, very high sustained usage, or complex connection state.

Trade-off: more operations and cost; unnecessary for a focused v1.

Two credentials, two different purposes

A user token answers “what may this agent do?”; an internal key answers “is this trusted infrastructure?”

1. User MCP token

Passed from MCP client to Lambda and forwarded to the Focus Bear API.

Validated as a raw opaque token; scope and agent assignment are checked server-side.

2. Internal service key

Stored in AWS Secrets Manager and read by the Lambda execution role.

Proves the request came through the approved adapter; rotate it without redeploying clients.

3. Existing policy checks

Focus Bear API enforces token scope, user ownership, and the token/agent assignment already recorded on a task.

No direct database access from Lambda.

Security gate before production

Eliminate hard-coded development fallbacks, require real authentication for token issuance, redact secrets in logs, and set explicit rate / concurrency limits.

A focused tool set keeps risk and delivery time low

Start with existing task operations; every write goes through the same Focus Bear API validation path.

List tasks

Query tasks assigned to the authenticated agent

Get task

Read a permitted task by ID

Update status

Move a permitted task to an allowed status

Add note

Append a task note through the API

Project statuses

List valid statuses for a project

Principle: expose only narrow, named business actions—not generic database access or arbitrary API proxying.

Designed for predictable behaviour under load

The adapter should fail safely and make both client and engineering diagnosis straightforward.

Capacity controls

Reserved concurrency protects the Focus Bear API; API Gateway throttling limits abusive traffic; short timeouts prevent request pile-up.

Error behaviour

Return protocol-safe errors. Do not retry writes automatically unless an idempotency key is in place. Preserve a correlation ID for support.

Observability

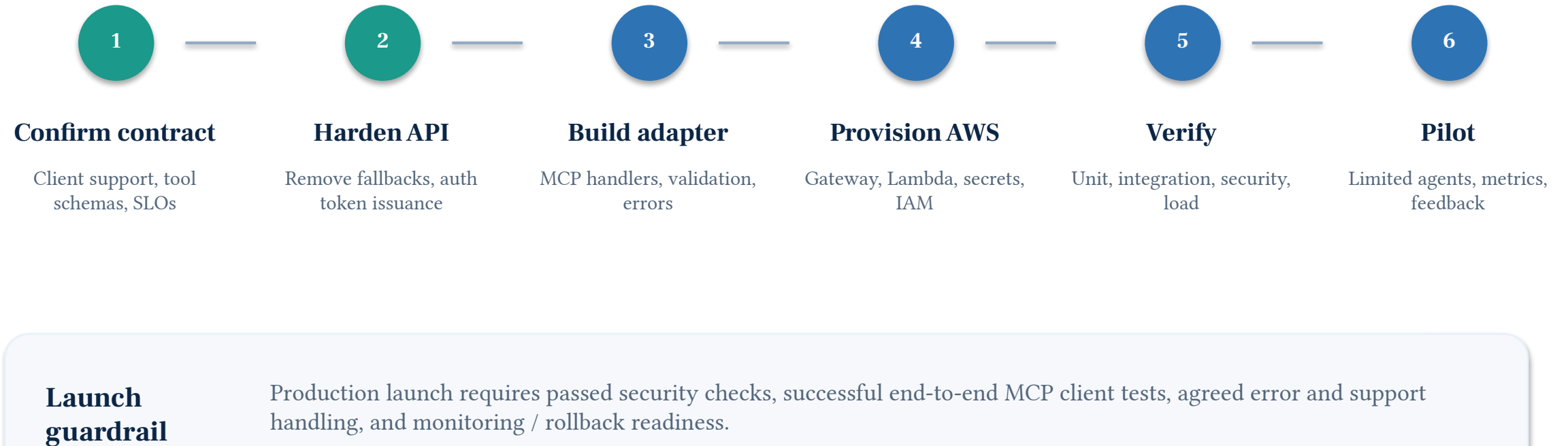
Track invocation volume, latency, error rate, throttles and cold starts. Alert on sustained failures; never log raw credentials or full task content by default.

Cost profile

Pay per request and execution time; no idle service cost. The main cost variables are request volume, streaming duration, logs/traces, and Secrets Manager use.

A staged rollout makes the risk visible and manageable

Exact duration follows the answers to the decision questions on the next slide.



What we need to confirm with Focus Bear

These answers turn the proposal into an implementation plan and a reliable estimate.

Target clients Which MCP clients must be supported—and do any need streamed responses or server-initiated messages?

Network path Where is the production Focus Bear API reachable from AWS, and how will the internal service credential be rotated?

Usage profile Expected agents, peak concurrent calls, request/response sizes, and target latency / availability?

Identity model Are opaque MCP tokens sufficient for v1, or is third-party OAuth / delegated identity required now?

Proposed next step: validate client transport requirements, then run a short technical discovery to lock the interfaces and launch controls.